01 Cover & Executive Summary

HEART DISEASE CLASSIFICATION END-TO-END MLOps WORKFLOW

Machine Learning Operations (MLOps) — Assignment 01

Course: AIMLCZG523 | Total marks: 50

Student: Jahanvi Punia | Enrollment ID: 2025AE05147

Submission date: 20 October 2026

Executive summary

This project develops and deploys a reproducible binary-classification workflow using the UCI Heart Disease dataset. The implementation covers dataset acquisition through ucimlrepo, data cleaning and exploratory analysis, preprocessing pipelines, cross-validation comparison of Logistic Regression and Random Forest, MLflow tracking, model packaging, automated lint/test/train jobs, Docker image building, a public FastAPI service on Render, and basic request logging/metrics.

Problem statement

Given 13 patient attributes, predict the dataset label for absence (0) or presence (1) of heart disease. The original target values 1–4 are combined into the positive class; the original zero label is mapped to absence.

Responsible-use statement

This is an educational prototype only. It is not clinically validated and must not be used for diagnosis, treatment, triage, or real-world patient decisions. The returned probability is a model score, not a calibrated clinical risk estimate.

Project links

- **Repository:** https://github.com/JahanviPunia/mlops-heart-disease-assignment
- **Live API docs:** https://heart-disease-mlops-api-8mg3.onrender.com/docs

What this submission demonstrates

The same fitted preprocessing-and-model bundle is used for offline evaluation and online prediction. GitHub Actions provides repeatable checks and training artifacts, while the public service makes the inference contract testable through a browser. The evidence in this report is tied to the submitted notebook, repository workflow runs, downloaded artifact archive and live endpoint checks.

02 Dataset Acquisition & Exploratory Data Analysis

Source and acquisition

The dataset is obtained from the UCI Machine Learning Repository using ucimlrepo dataset ID 45. The submitted Colab notebook includes the fetch script, data inspection, cleaning, and saving of a cleaned CSV. The inspected data has 303 records and 13 input features, plus target information.

Input variables

age, sex, cp, trestbps, chol, fbs, restecg, thalach, exang, oldpeak, slope, ca, and thal. The target is converted to a binary field: original num=0 becomes 0; original num>0 becomes 1.

Data quality and class balance

The notebook's inspection found 4 missing values in ca and 2 in thal. The binary class counts are 164 records for absence and 139 for presence. Missing values are handled in the fitted preprocessing pipeline so transformations can be reproduced at inference time.

EDA evidence

The notebook generates class-balance, numeric feature-distribution, and correlation visualizations and inspects missing values. The charts below are extracted from the submitted Colab notebook.

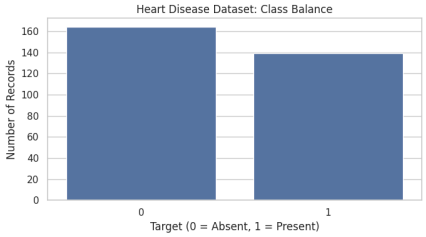


Figure 1. Class balance visualization from the submitted notebook.

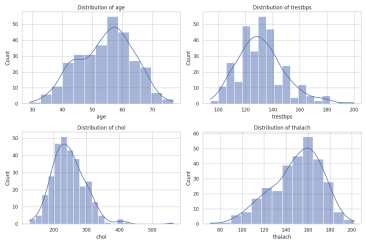


Figure 2. Feature distribution visualization from the submitted notebook.

The dataset is small and its observed class balance does not establish representativeness of wider patient populations.

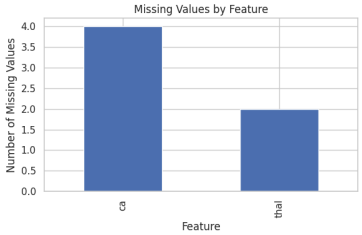


Figure 3. Additional EDA chart output from the submitted notebook.

03 Feature Engineering & Model Development

Preprocessing

The scikit-learn pipeline separates numeric variables (age, trestbps, chol, thalach, oldpeak) from categorical-coded variables (sex, cp, fbs, restecg, exang, slope, ca, thal). Numeric variables use median imputation and StandardScaler. Categorical-coded variables use most-frequent imputation. The fitted preprocessing and classifier are stored together in the model bundle.

Models and validation

Two classifiers were evaluated: Logistic Regression (max_iter=2000, class_weight='balanced', random_state=42) and Random Forest (250 estimators, min_samples_leaf=2, class_weight='balanced', random_state=42). Evaluation uses stratified five-fold cross-validation with accuracy, precision, recall, and ROC-AUC.

Model	Accuracy	Precision	Recall	ROC-AUC
Logistic Regression	0.8349	0.8319	0.8056	0.9099
Random Forest	0.8416	0.8602	0.7839	0.9078

Selection rationale

Logistic Regression was selected by the predefined mean ROC-AUC criterion: 0.9099 versus 0.9078 for Random Forest. Random Forest had higher mean accuracy and precision; Logistic Regression had higher recall and ROC-AUC. The difference in ROC-AUC is small, so this is a baseline choice rather than evidence of clinical superiority.

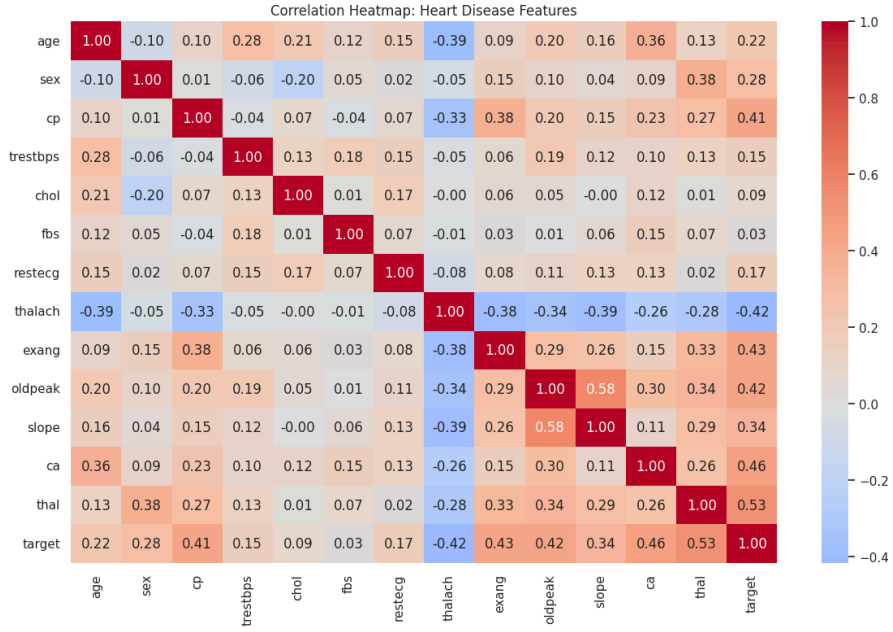


Figure 1. Additional feature/correlation visualization from the notebook.

The current pipeline imputes categorical-coded variables but does not one-hot encode them. This is a limitation and a potential future improvement.

Evaluation methodology

StratifiedKFold was configured for five folds so each fold maintains a similar target-class proportion. The recorded metrics are fold-averaged estimates. Accuracy summarizes overall correct classifications; precision measures the share of predicted-positive cases that were positive in the labels; recall measures the share of positive-labelled cases identified; ROC-AUC summarizes ranking discrimination across thresholds.

Why not select by accuracy alone?

The assignment asks for multiple relevant metrics. Logistic Regression's mean accuracy is slightly lower than Random Forest's, but its ROC-AUC is slightly higher and its recall is also higher. Because the training code explicitly chooses the highest mean ROC-AUC, the selected model follows a declared criterion. No extensive hyperparameter search was performed; the parameters listed above are the configured baseline settings.

Feature treatment note

StandardScaler is applied to the five numeric fields. The remaining fields are retained as numeric-coded categories and imputed using the most frequent value. Since integer category codes can imply artificial ordering to some estimators, one-hot encoding or model-specific categorical handling would be a sensible next experiment.

04 Experiment Tracking & Reproducibility

MLflow

The notebook sets up MLflow with experiment name Heart-Disease-Classification and a local SQLite tracking URI. Candidate runs record model parameters and evaluation metrics. The notebook reports four experiment runs. The SQLite database is included in the CI artifact ZIP.

Saved model and reports

- artifacts/model.joblib — fitted pipeline, feature-column metadata and selected model name.
- reports/model_comparison.csv — mean cross-validation metrics for both candidate models.
- reports/training_summary.json — model and training summary.
- reports/class_balance.png — saved EDA class-balance plot.
- mlflow.db — local MLflow tracking database from the training run.

Reproducible environment

The pinned requirements file records the initial Colab environment: Python 3.13.16, pandas 2.2.3, NumPy 2.1.3, scikit-learn 1.6.1, joblib 1.6.0, MLflow 3.17.1, FastAPI 0.141.1 and Pydantic 2.14.1. The training script reads data/heart_disease_cleaned.csv and writes the model and report artifacts.

Artifact verification

GitHub Actions uploaded mlops-training-artifacts successfully. The downloaded ZIP was inspected: model.joblib was present under artifacts/, and class_balance.png, model_comparison.csv and training_summary.json were present under reports/.

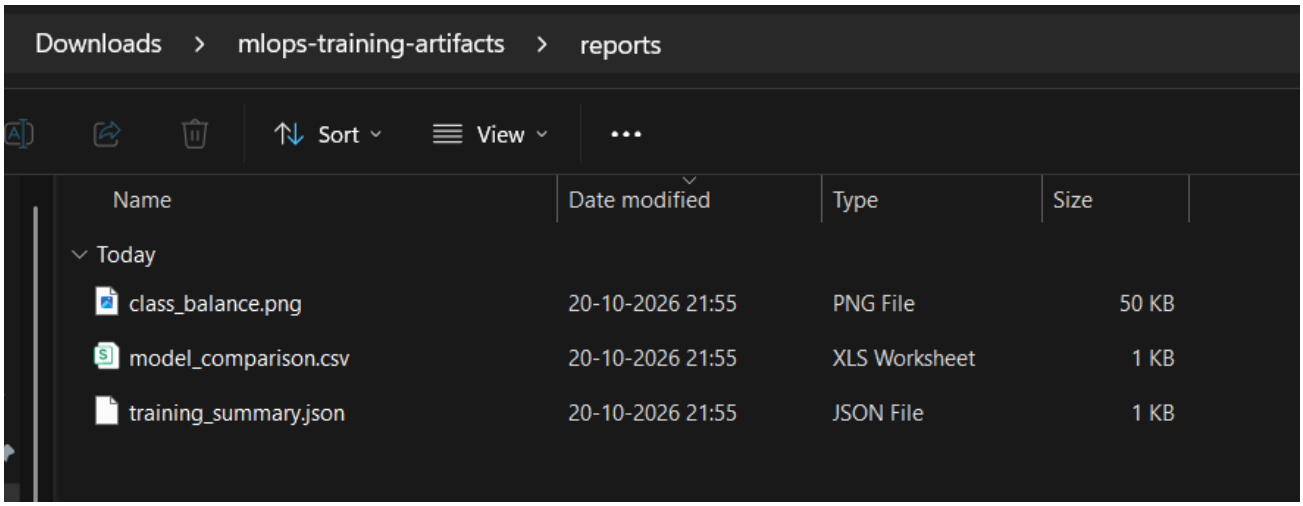


Figure 1. Verified report files inside the downloaded CI artifact.

05 API, Containerization & Architecture

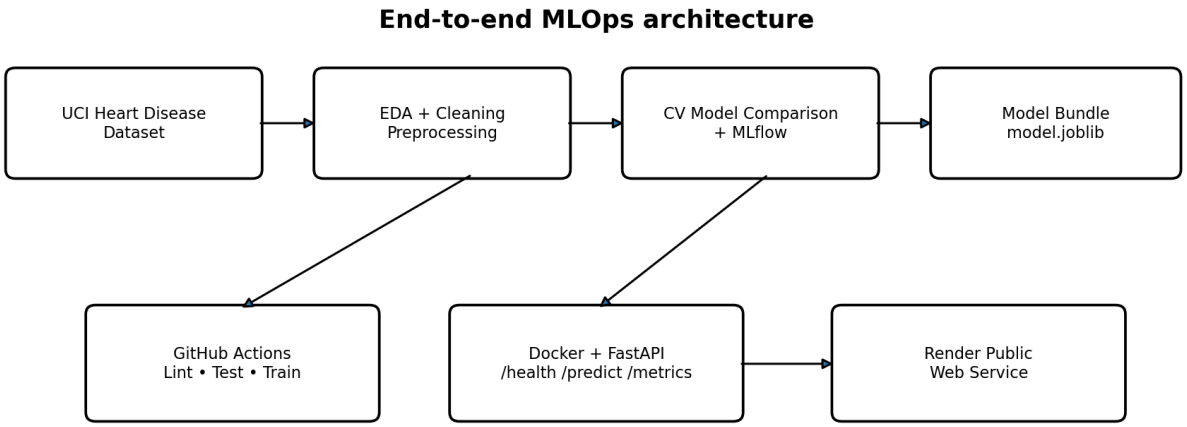


Figure 1. Project architecture and lifecycle.

FastAPI endpoints

Endpoint	Function	Observed result
GET /health	Checks service and loaded model	HTTP 200; status=ok; model_loaded=true
POST /predict	Receives JSON features; returns class, label and probability	HTTP 200; Logistic Regression response verified
GET /metrics	Returns an in-memory request counter	Returned api_requests_total=83 during verification

Input and response

The request schema defines the 13 feature fields and rejects unexpected fields. The tested sample returned prediction=0, label=disease_absent, positive_class_probability=0.0259, model=Logistic Regression, plus an educational-only disclaimer.

Docker packaging

The Dockerfile uses python:3.13-slim, installs requirements.txt, copies src/ and artifacts/, and starts Uvicorn on 0.0.0.0. The deployed Render service is listening successfully on port 10000. The separate GitHub Actions Docker Build workflow passed.

Testing

The API test suite covers health, a valid prediction, rejection of unexpected fields, and the metrics endpoint. The CI workflow runs pytest against tests/test_api.py. Dedicated unit tests for standalone data-processing/model utility functions are not currently evidenced and remain a potential enhancement.

Request contract

POST /predict expects a JSON object with age, sex, cp, trestbps, chol, fbs, restecg, thalach, exang, oldpeak, ca and thal, with the two latter fields permitted to be null for missing values. Extra fields are rejected by the Pydantic request schema. The server reorders the request columns to the feature order stored in the model bundle before prediction.

Operational behavior

The service loads artifacts/model.joblib at process startup. /health reports whether the model is loaded; /predict returns a class, readable label, positive-class probability, model name and disclaimer; /metrics returns a simple request count. Middleware logs request method, route, status and duration, and sets the X-Process-Time-ms header. The counter is intentionally simple and process-local, not a durable monitoring system.

Container verification boundary

The GitHub Docker Build workflow passed, and the deployed Render service successfully served the health and prediction endpoints. A separate local Docker run was not performed in the browser-only setup, so this report does not claim a local runtime test.

06 CI/CD & Automated Artifact Evidence

GitHub Actions workflow

The MLOps CI workflow has three jobs: lint, test and train. The latest observed run, MLOps CI #5, shows all three jobs successful. The separate Docker Build workflow's latest observed run, Docker Build #4, is also successful.

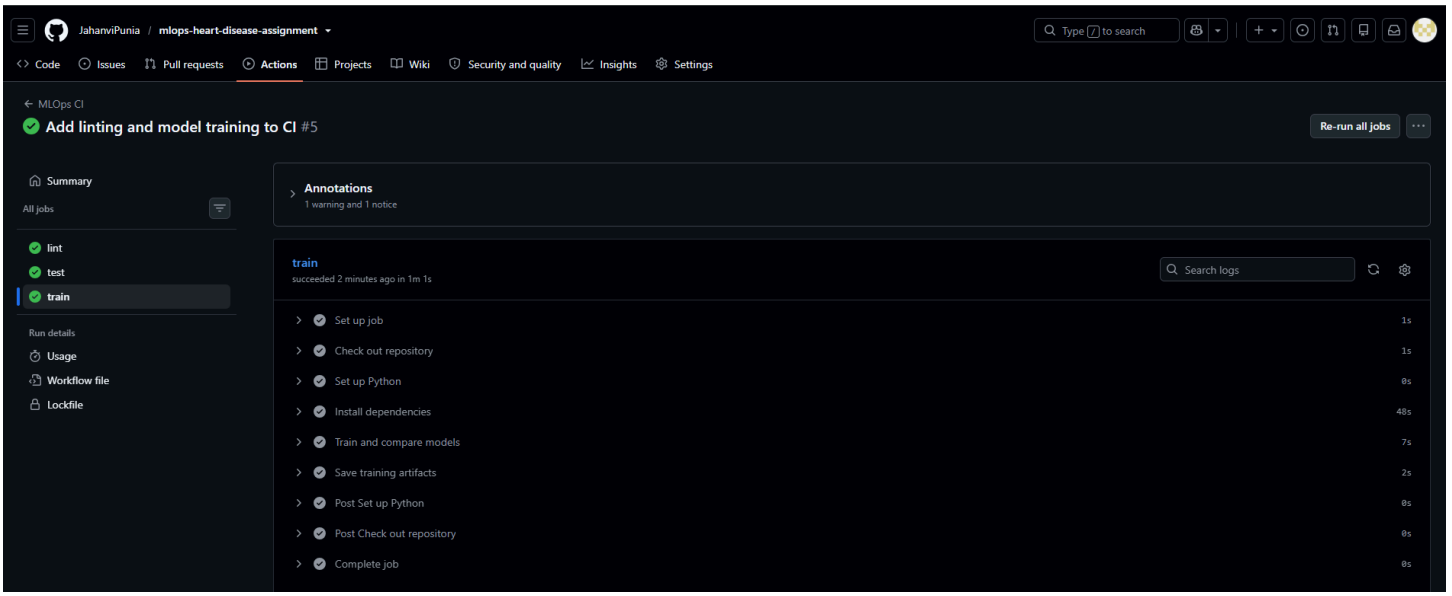


Figure 1. Latest MLOps CI run: lint, test and train all succeeded.

Training and artifact upload

The training job ran the model comparison and completed the Save training artifacts step. The upload log confirms five files uploaded, a 63,611-byte archive, and artifact name mlops-training-artifacts. The archive was downloaded and its contents were checked.

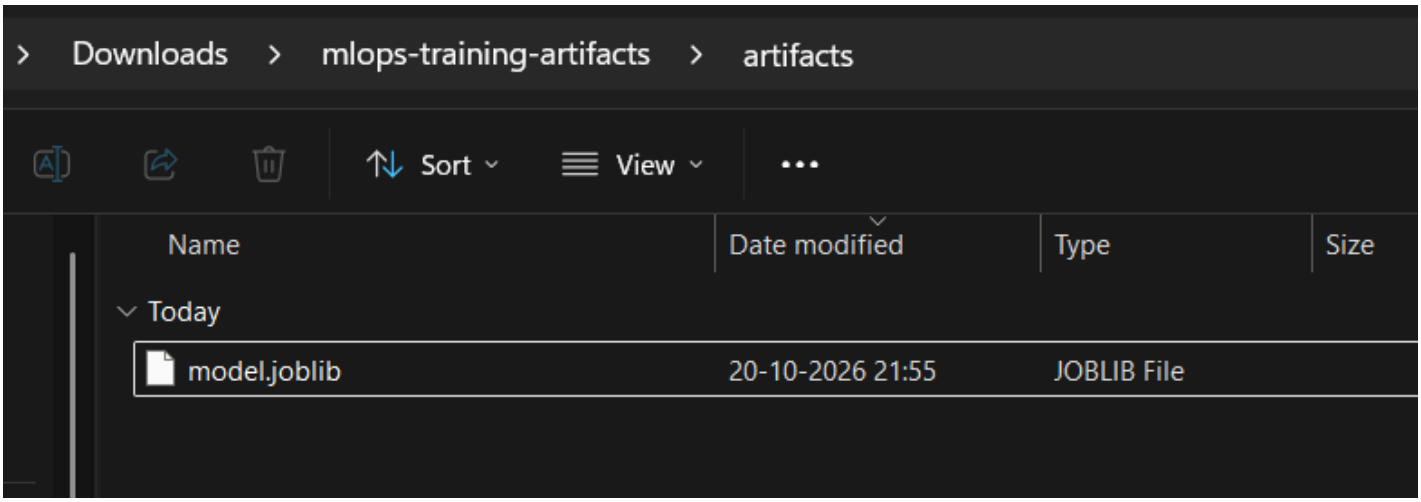


Figure 2. Trained model artifact present in the downloaded ZIP.

Workflow behavior and boundaries

The CI pipeline fails when lint, tests or training commands fail and uploads artifacts after successful training. Docker Build verifies that the image builds. CI does not automatically promote its newly built image to Render; Render's deployment is managed separately.

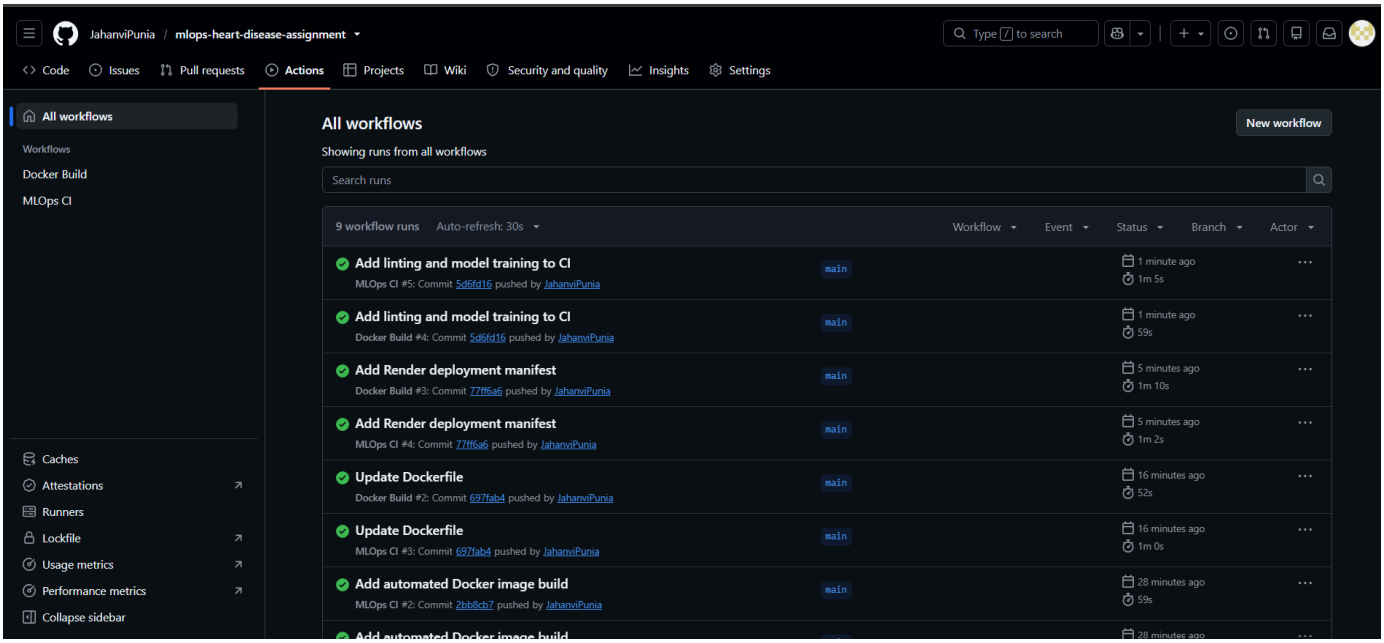


Figure 3. GitHub Actions history showing successful workflow runs.

07 Public Deployment & Endpoint Verification

Live service

The Dockerized FastAPI application is deployed as a Render Web Service named heart-disease-mlops-api. The public base URL is https://heart-disease-mlops-api-8mg3.onrender.com. The free instance can spin down after inactivity and may take longer to respond after a period of inactivity.

Verified links

- Interactive API docs: https://heart-disease-mlops-api-8mg3.onrender.com/docs
- Health: https://heart-disease-mlops-api-8mg3.onrender.com/health
- Metrics: https://heart-disease-mlops-api-8mg3.onrender.com/metrics
- Repository: https://github.com/JahanviPunia/mlops-heart-disease-assignment
- Actions: https://github.com/JahanviPunia/mlops-heart-disease-assignment/actions

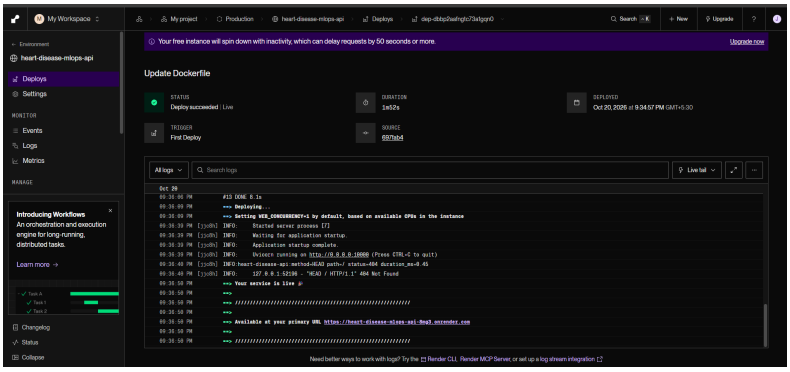


Figure 1. Render deployment marked Deploy succeeded / Live.

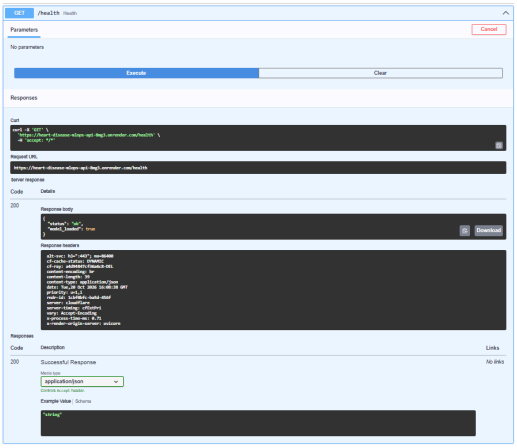


Figure 2. Public health endpoint returned HTTP 200 and model_loaded=true.

Deployment manifest

The repository includes render.yaml documenting the web service name, Docker runtime, Dockerfile path, free plan and /health health check. The service was created and verified in the Render dashboard; the manifest documents configuration but was not independently verified as the mechanism that provisioned the live service.

08 Prediction Evidence & Monitoring

Prediction endpoint

A sample request was submitted using the interactive Swagger documentation. The service returned HTTP 200 and a JSON response containing prediction, label, positive-class probability, model name and the educational disclaimer.

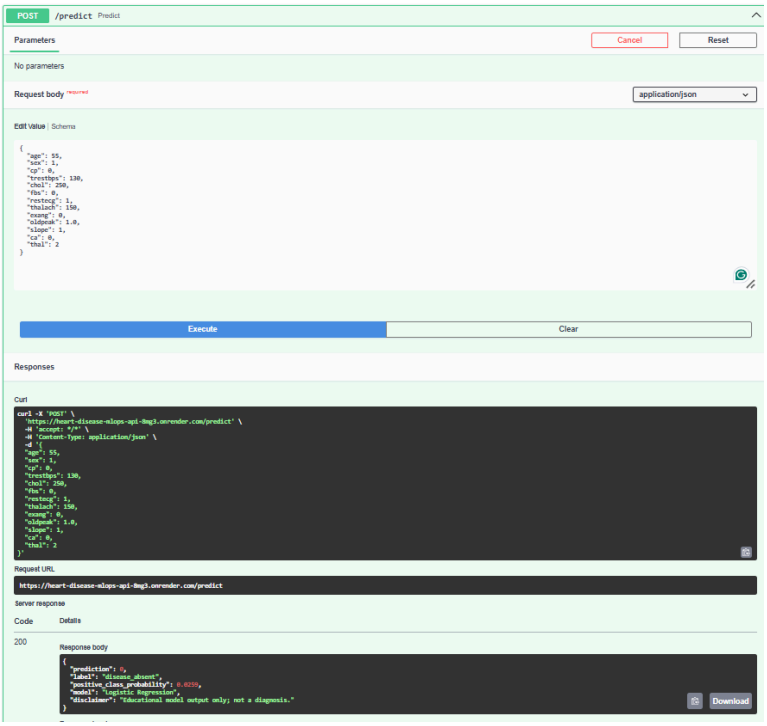


Figure 1. POST /predict sample request and response.

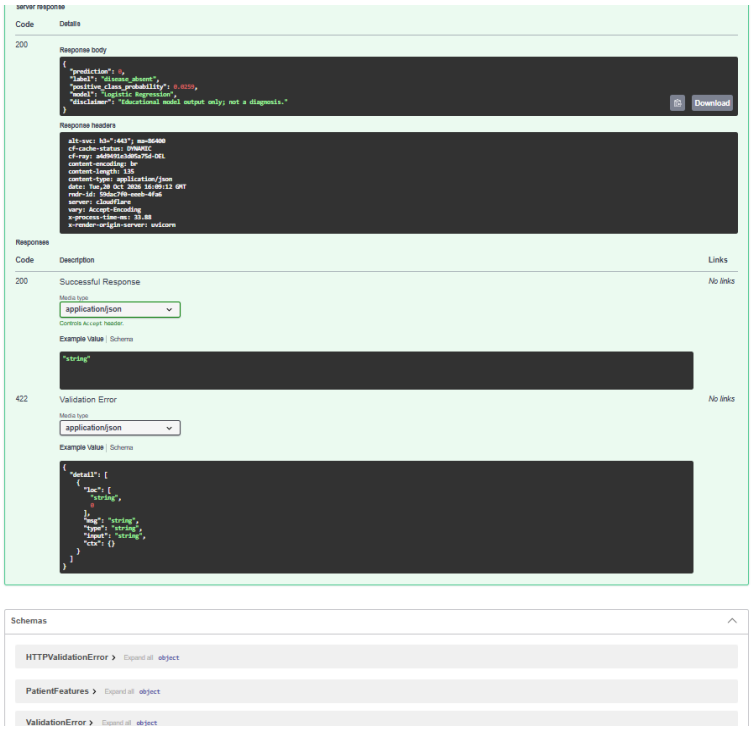


Figure 2. Prediction response fields and validation-error schema.

Application logs and request metrics

Middleware logs request method, path, status code and duration in milliseconds. Responses include the X-Process-Time-ms header. GET /metrics exposes an in-memory api_requests_total counter, which returned 83 during verification.

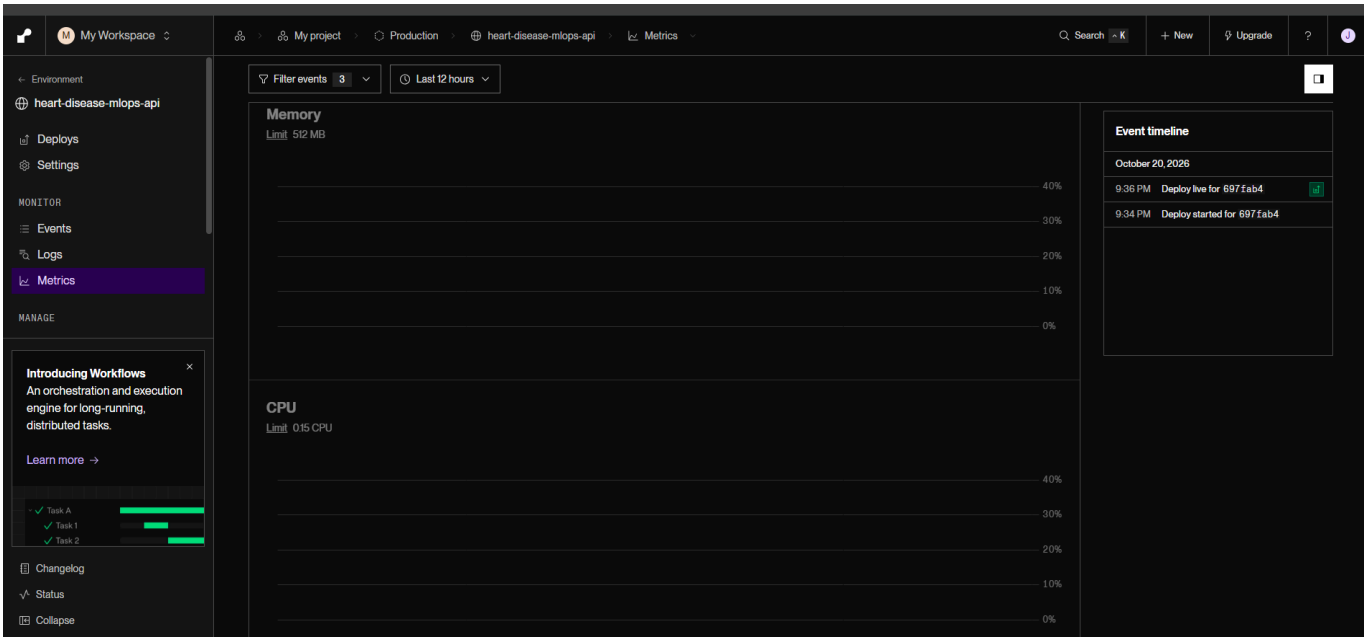


Figure 3. Render Metrics screen; limits are shown but detailed utilization requires a paid plan.

Monitoring limitation

The free Render plan displayed limits of 512 MB memory and 0.15 CPU but did not expose actual CPU/memory utilization graphs. The request counter is in-memory and may reset on restart or spin-down. No persistent Prometheus/Grafana dashboard has been configured.

> *Do not interpret the model output as a medical recommendation. The service has not undergone clinical validation, calibration, fairness evaluation, privacy/security review or regulatory assessment.*

09 Setup, Limitations & Final Submission Checklist

Reproduction instructions

Environment: Python 3.13. Install the dependencies with pip install -r requirements.txt. Ensure data/heart_disease_cleaned.csv exists. Run training with python src/train.py; run API tests with pytest -v tests/test_api.py; build the image with docker build -t heart-disease-api .; then run the container and check /health, /predict and /metrics. The public Render deployment and GitHub Actions runs provide cloud-based verification. A local Docker runtime was not available in the browser-only workflow, so local container execution is not claimed as personally verified.

Known limitations and next improvements

- Dataset contains 303 records; no external validation was performed.
- Categorical-coded features are not one-hot encoded in the current pipeline.
- Add unit tests for data-processing and model utility functions.
- Add persistent metrics/dashboarding (for example, Prometheus/Grafana) if available.
- Consider model calibration, subgroup evaluation, and external validation before any clinical use.

Deliverables checklist against the brief

- [x] Dataset acquisition instructions, cleaned dataset and EDA visualizations in notebook/project.
- [x] Two models, cross-validation metrics and model-selection rationale.
- [x] MLflow tracking and serialized model artifact.
- [x] Requirements, preprocessing pipeline and training script.
- [x] GitHub Actions lint/test/train workflow and artifact upload.
- [x] Dockerfile and successful Docker image build workflow.
- [x] Public Render service, manifest, endpoint tests and screenshots.
- [x] API request logs, request-time header and basic /metrics counter.
- [x] Ten-page report with architecture, links and screenshots.
- [ ] Record and attach a short end-to-end video, required separately by the assignment.

References

UCI Machine Learning Repository — Heart Disease dataset, accessed using ucimlrepo dataset ID 45. Assignment: Machine Learning Operations (MLOps) AIMLCZG523, Assignment 01. Code and workflow: https://github.com/JahanviPunia/mlops-heart-disease-assignment. Live API docs: https://heart-disease-mlops-api-8mg3.onrender.com/docs.

10 Evidence Register & Submission Links

Source code and public deployment

- GitHub repository: https://github.com/JahanviPunia/mlops-heart-disease-assignment
- GitHub Actions runs: https://github.com/JahanviPunia/mlops-heart-disease-assignment/actions
- Live API documentation: https://heart-disease-mlops-api-8mg3.onrender.com/docs
- Health endpoint: https://heart-disease-mlops-api-8mg3.onrender.com/health
- Metrics endpoint: https://heart-disease-mlops-api-8mg3.onrender.com/metrics

Evidence included with this report

- EDA chart(s) extracted from the submitted Colab notebook.
- Render deployment status screenshot.
- Public health endpoint response screenshot.
- Swagger prediction request and response screenshots.
- Render Metrics screenshot showing the free-tier limitation.
- GitHub Actions history and MLOps CI job summary screenshots.
- Downloaded CI artifact screenshots showing model.joblib and report outputs.